

DEEP LEARNING CAPSTONE

CIFAR-10 with Dense Networks Only

How far can a fully connected network see, with no convolution allowed?



3,072

inputs per image



20

experiments



52.94%

test accuracy



58.36%

best with augmentation

Khalid - Wasan

Building & Developing AI Models Bootcamp

One rule shapes everything

- No CNN, no Conv2D, no pretrained weights, no transfer learning

● The task

CIFAR-10, ten mutually exclusive classes

60,000 colour images at $32 \times 32 \times 3$

Softmax output, sparse categorical cross-entropy loss

● The rule

Dense layers only

Every image must be flattened before it reaches the first layer

$32 \times 32 \times 3$ becomes $3,072 \times 1$

● What that costs

One weight per pixel position

No translation invariance: shift an object three pixels and almost every input value changes

The network must learn the same object again

- That single decision sets the ceiling for every experiment that follows.

The data

- One row is one 32 x 32 colour image, ten balanced classes

One example of each of the 10 classes



One example of each class

15 random training images

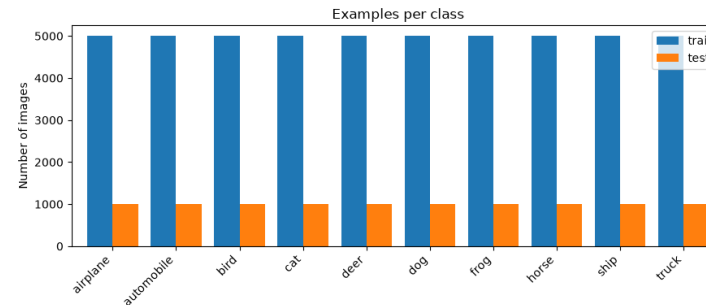
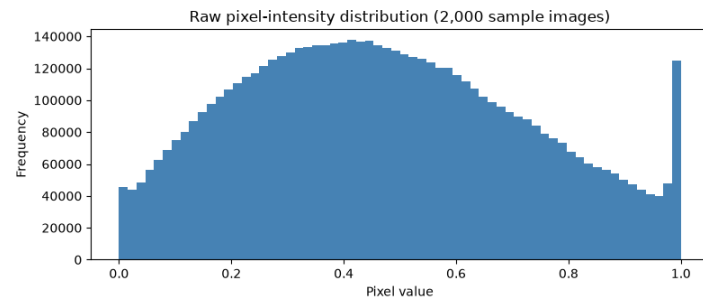
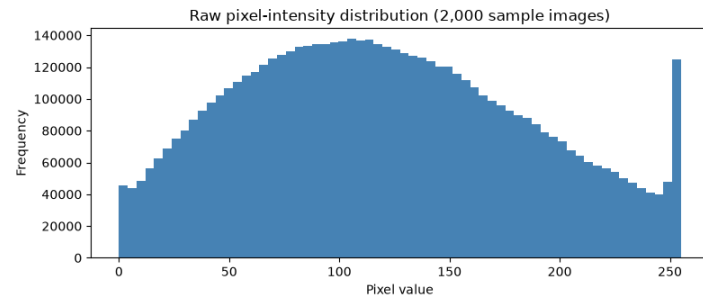


Fifteen random training images

Ten mutually exclusive classes at 32 x 32 pixels. At this resolution the animal classes already look alike.

Preparation

- Scaled, split once, and committed so every experiment shares it



Exactly 6,000 images per class

● The split

45,000 train
5,000 validation, stratified
10,000 test, untouched until
Part 16

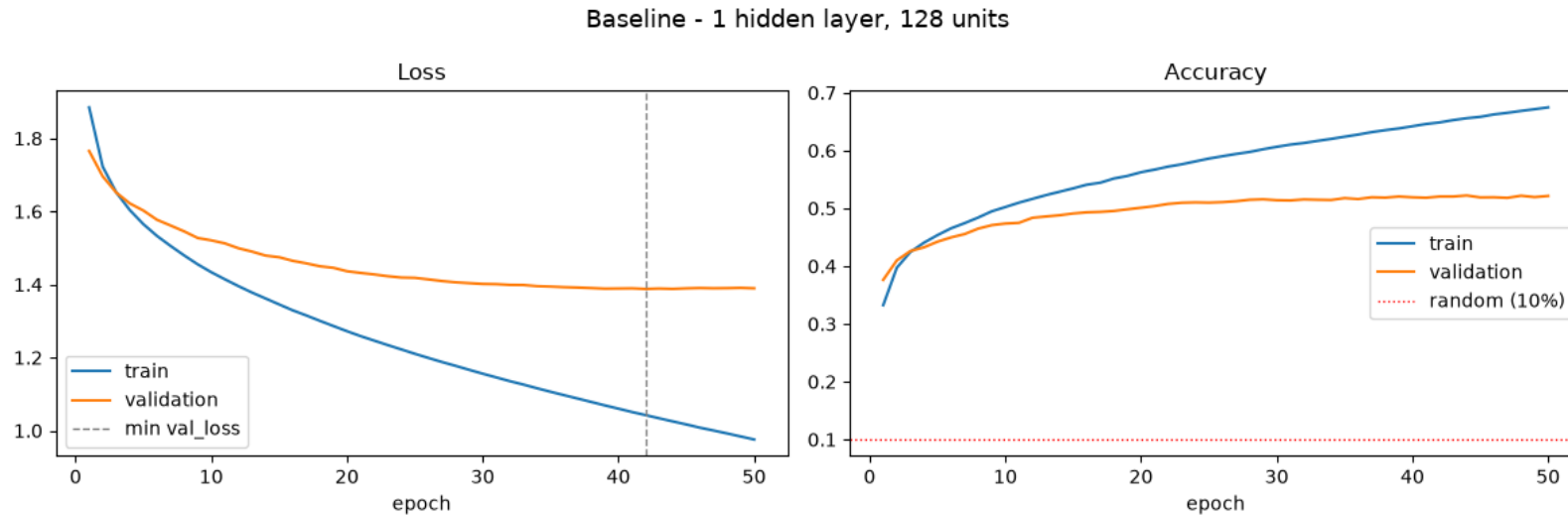
Pixels scaled to $[0, 1]$

Indices saved to splits.npz and
committed to git

● **The test set is opened once, at the very end. Every tuning decision is made on validation only.**

Part 3 - The baseline

- One hidden layer, 512 units, Adam at 0.0001, batch 128



●
52.22%

best validation accuracy

●
1,578,506

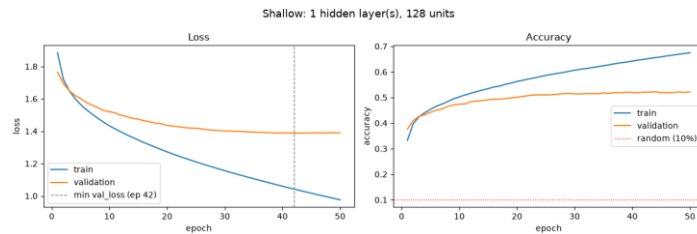
trainable parameters

● About 5x the 10% random-guess floor, and 99.7% of those parameters sit in the very first layer.

Parts 4-6 - Three depths, one width

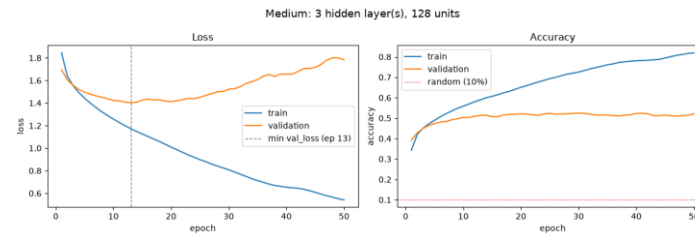
- Same data, same seed, same hyperparameters, only depth changes

Shallow 1 x 512



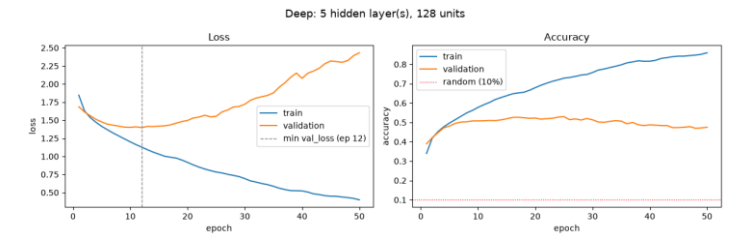
52.22% gap 0.15

Medium 3 x 512



52.40% gap 0.30

Deep 5 x 512

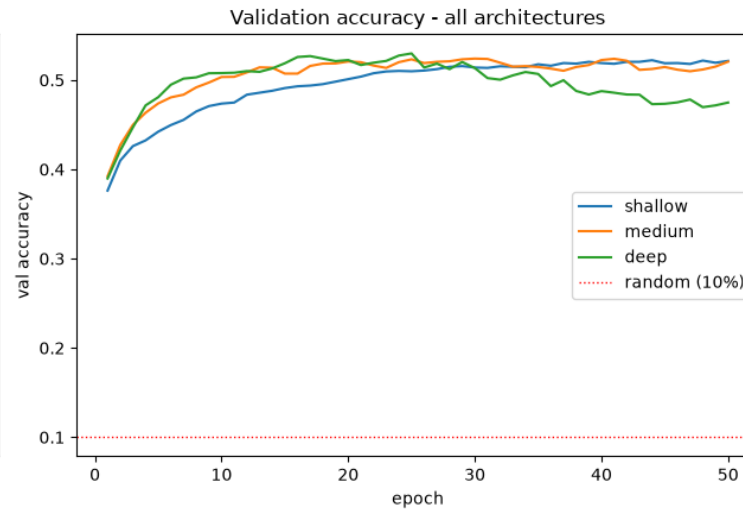
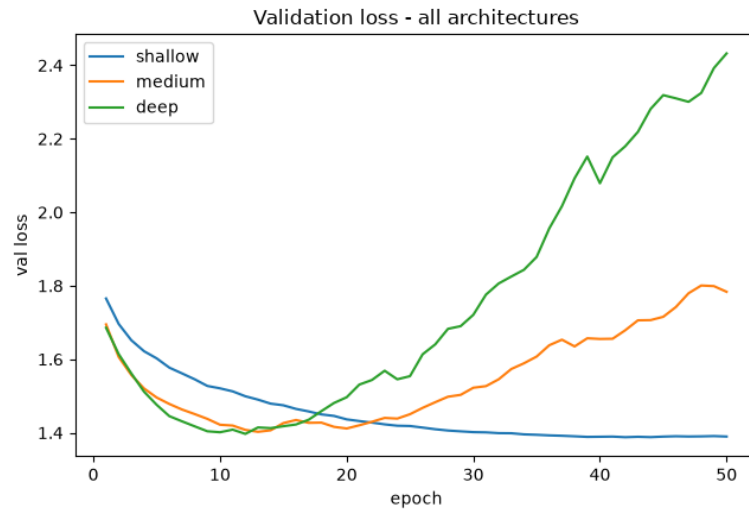


52.98% gap 0.33

Depth barely moves validation accuracy, but it steadily widens the gap between training and validation.

Part 7 - The comparison

- All three land within 38 images of each other



● **52.22%**

Shallow gap 0.15

● **52.40%**

Medium gap 0.30

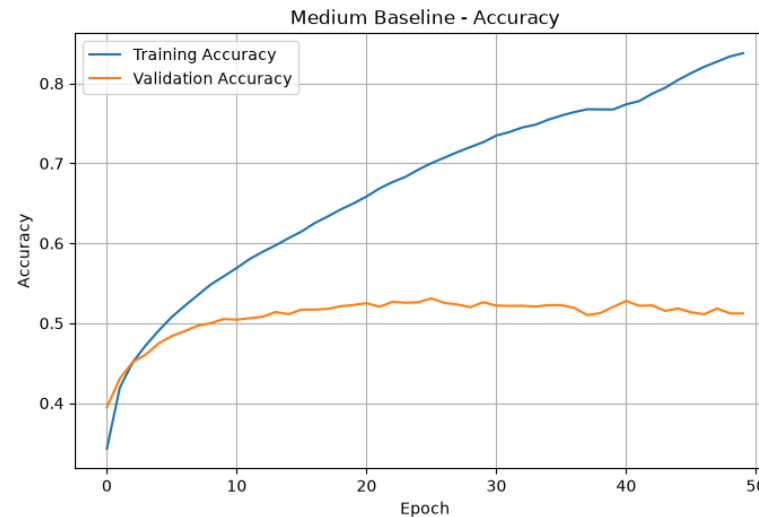
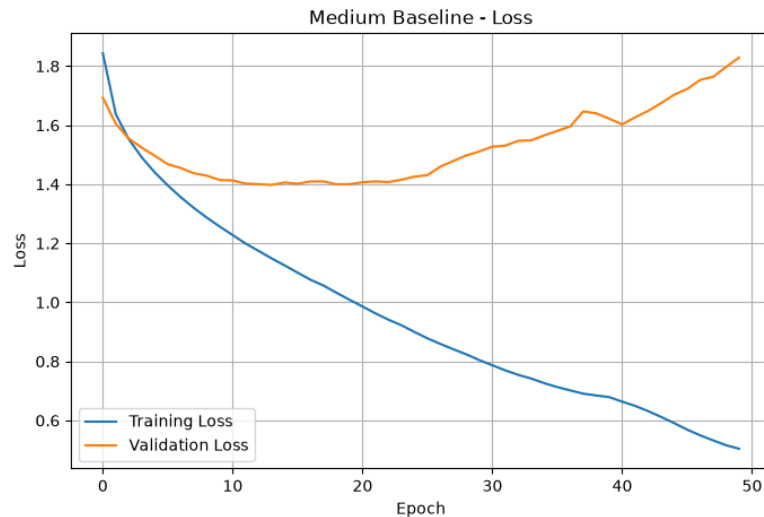
● **52.98%**

Deep gap 0.33

● 0.76 points apart on a 5,000-image validation set is inside the noise. Depth bought overfitting, not accuracy.

Part 8 - The diagnosis

● Medium, 3 x 512, 2,103,818 parameters



● **Overfitting, from epoch 13**

Training loss falls all 50 epochs,
1.845 to 0.542

Validation loss bottoms at
1.4024
then rises 27% to 1.7835

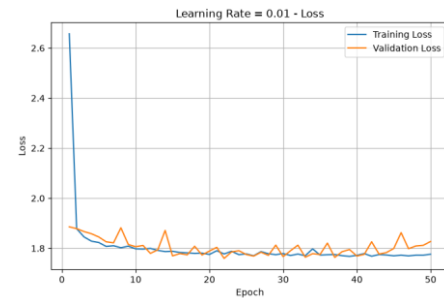
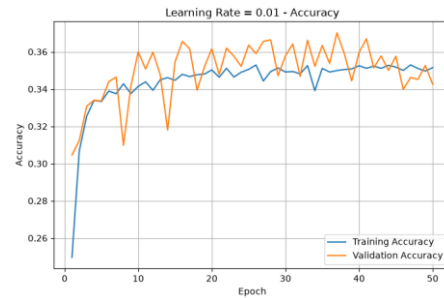
Training accuracy reaches
0.820
and is still climbing

Fig. 5.1. Stopping point

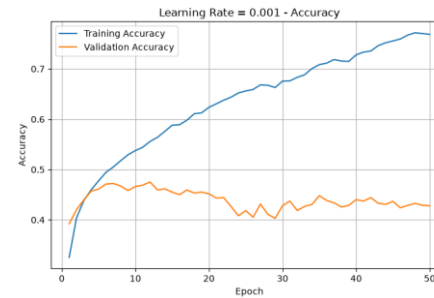
● The gap widens from 0.0759 at the best epoch to 0.2993 by epoch 50.

Part 9 - Learning rate

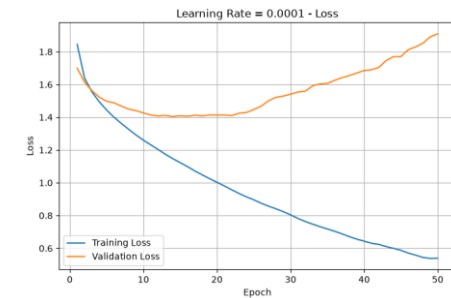
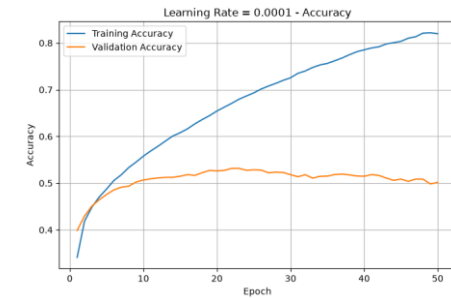
- 0.01 / 0.001 / 0.0001, everything else held fixed



0.01 -> 37.02%



0.001 -> 47.52%

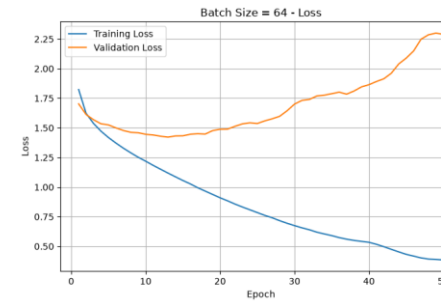
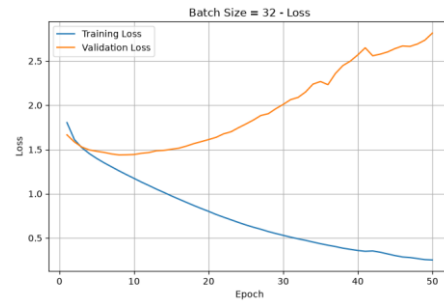
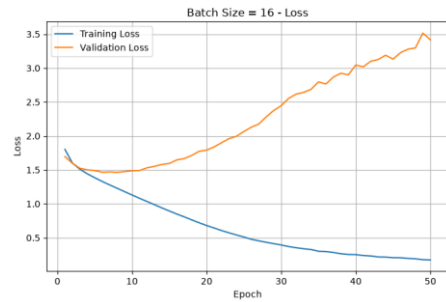
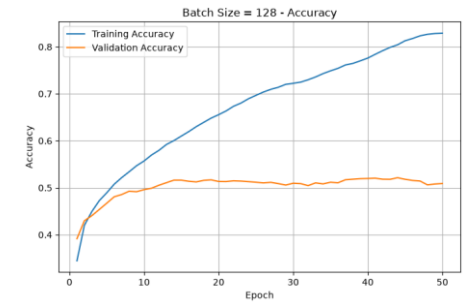
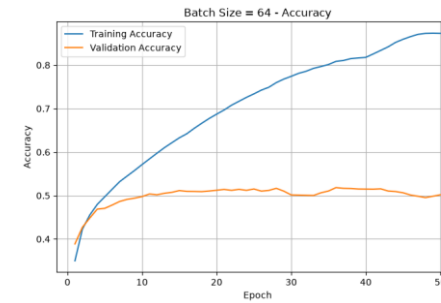
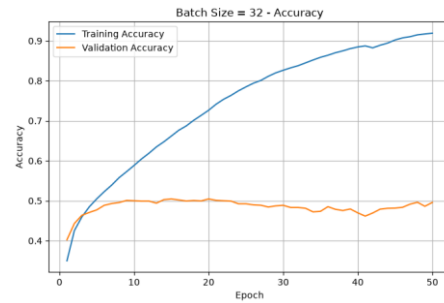
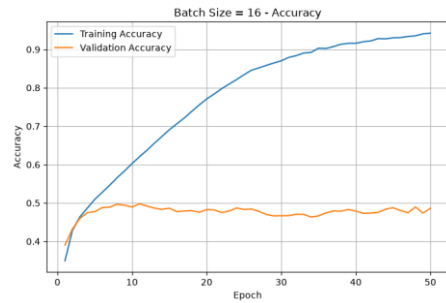


0.0001 -> 53.16%

0.0001 wins on both accuracy and loss. At 0.001 the model peaks by epoch 12 then diverges to a loss of 3.13.

Part 10 - Batch size

● 16 / 32 / 64 / 128 at the selected learning rate



16 -> 49.78%

32 -> 50.46%

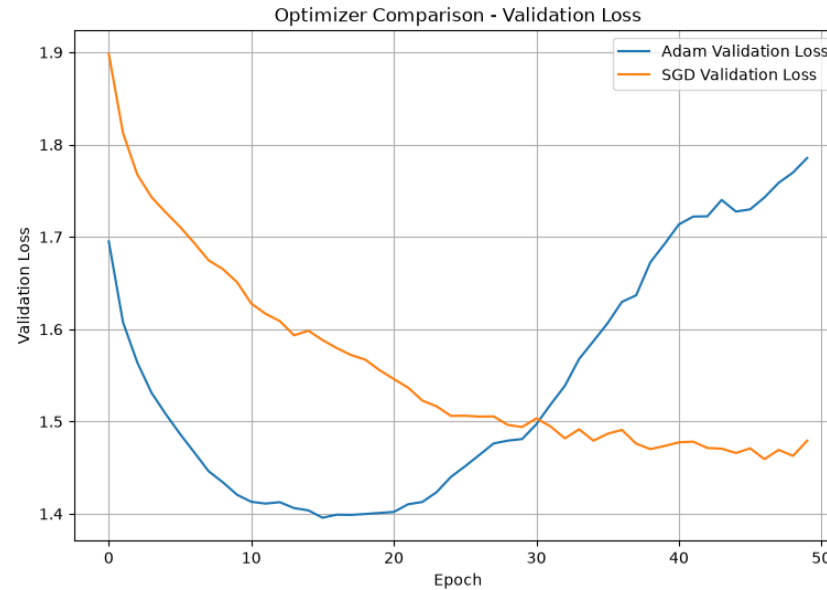
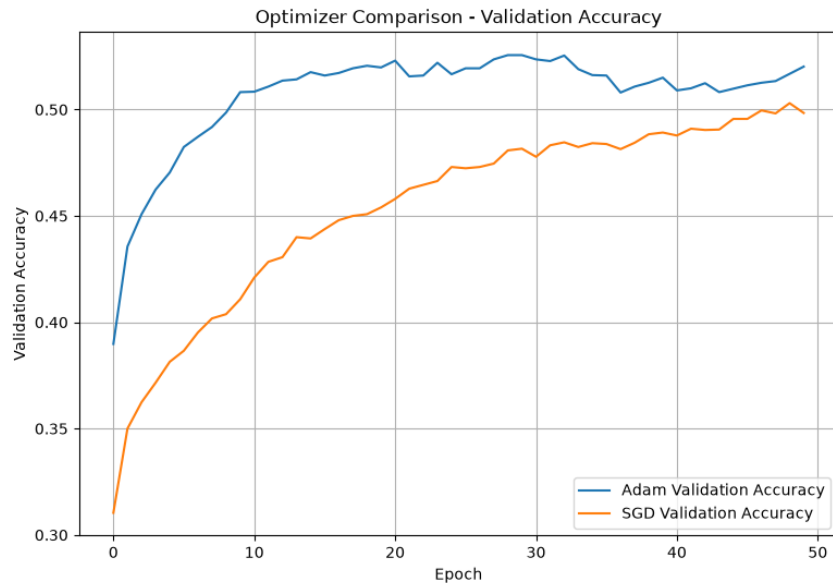
64 -> 51.82%

128 -> 52.14%

● Accuracy improved at every step. Batch 16 reached 94% training accuracy and a validation loss of 3.42.

Part 11 - Optimizer

Adam at 0.0001 against SGD at 0.01



Adam wins

52.56% against
50.30%

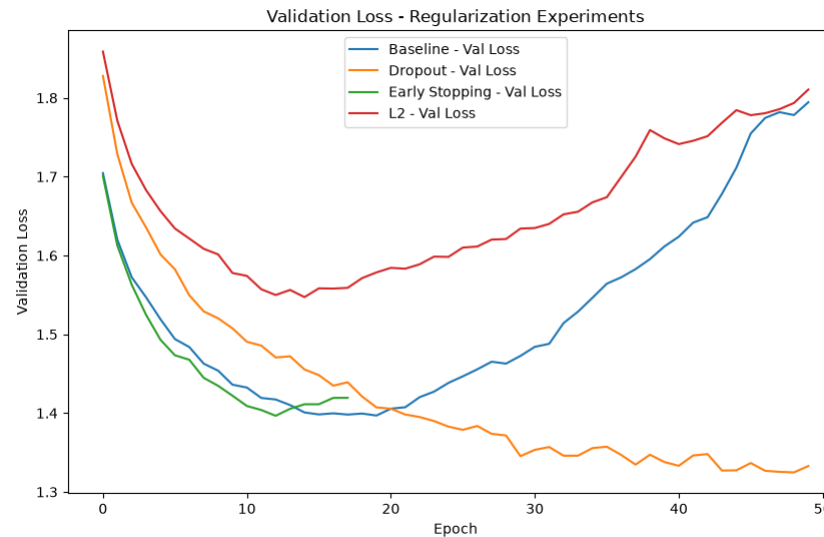
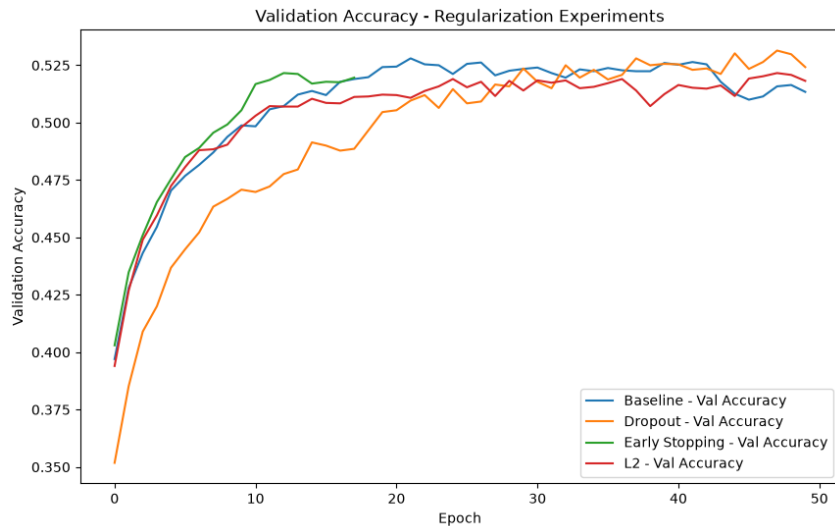
Min val loss 1.3957
against 1.4592

Peaks at epoch 29;
SGD is still climbing
at epoch 49

The learning rates differ on purpose: Adam normalises its steps, so matched rates would cripple SGD.

Part 12 - Regularization

Dropout 0.3, Early Stopping and L2 against an unregularized baseline



Dropout wins

52.42% accuracy

Gap 29.96% ->
6.17%

Val loss 1.3328,
the lowest of the four

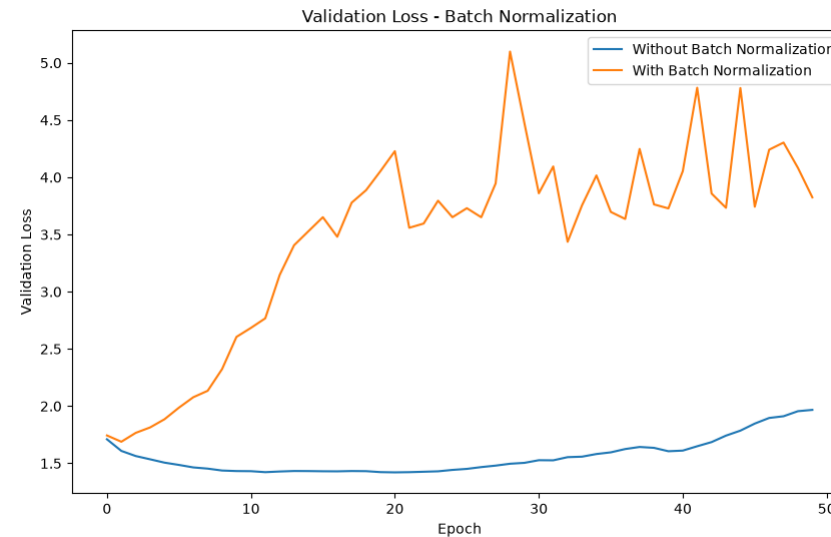
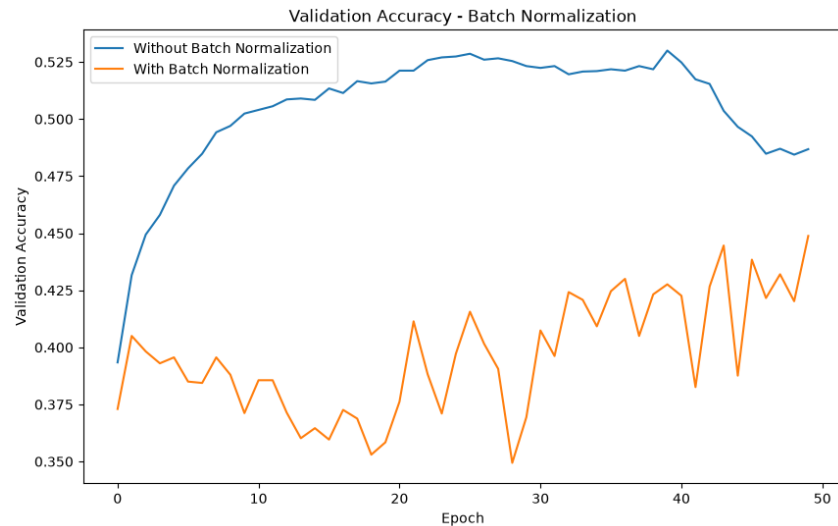
Early Stopping
51.96%

L2 51.82%

Dropout is the only technique that closes the gap rather than trimming it. L2 barely moves it.

Part 13 - Batch Normalization

- The same network, with and without BatchNorm after each Dense layer



- **It backfired**

Train 99.13%

Validation 44.88%

Gap 54.25%

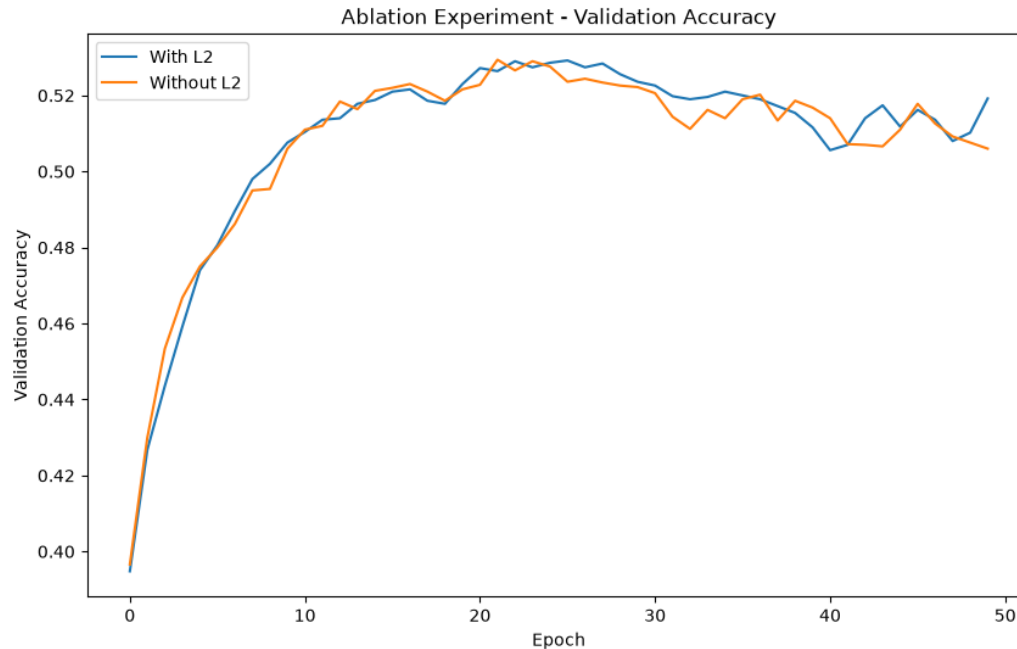
Final val loss 3.8244

Best val accuracy fell
from 53.00% to
44.88%

- The clearest demonstration in the project that high training accuracy is not evidence of a good model.

Part 14 - Ablation

- One variable removed, everything else identical



- L2 helps the ending

Final validation accuracy 51.92% against 50.60%

Gap 27.72% against 31.95%

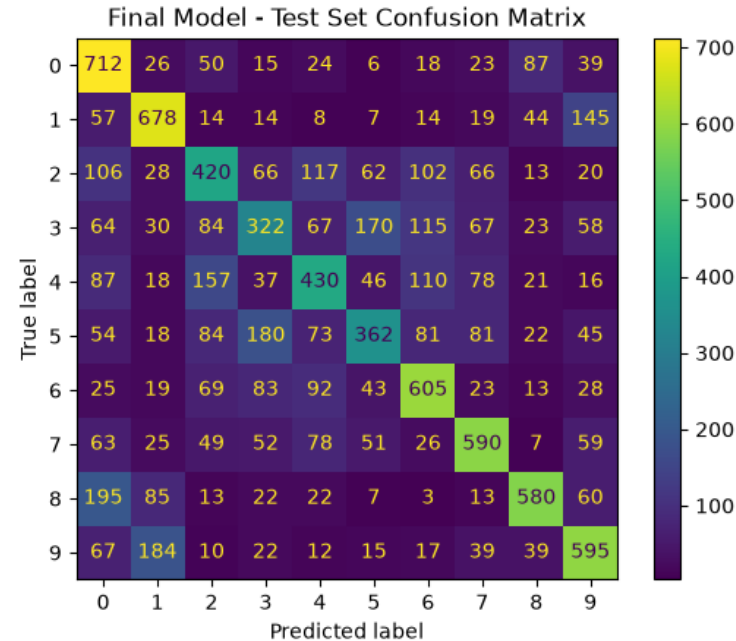
- But not the peak

Best validation accuracy 52.92% with L2, 52.94% without: one image out of 5,000

● L2 limits how far the model overfits by epoch 50. It does not raise the best result the model reaches.

Parts 15-16 - Final model and test

- 3 x 512, Adam 0.0001, batch 128, L2 0.0001, Early Stopping



Per-class F1

automobile 0.64 frog 0.58
ship 0.63 truck 0.58
horse 0.59 deer 0.45
airplane 0.59 bird 0.43
dog 0.41
cat 0.36

52.94
%
test accuracy

52.43
%
weighted F1

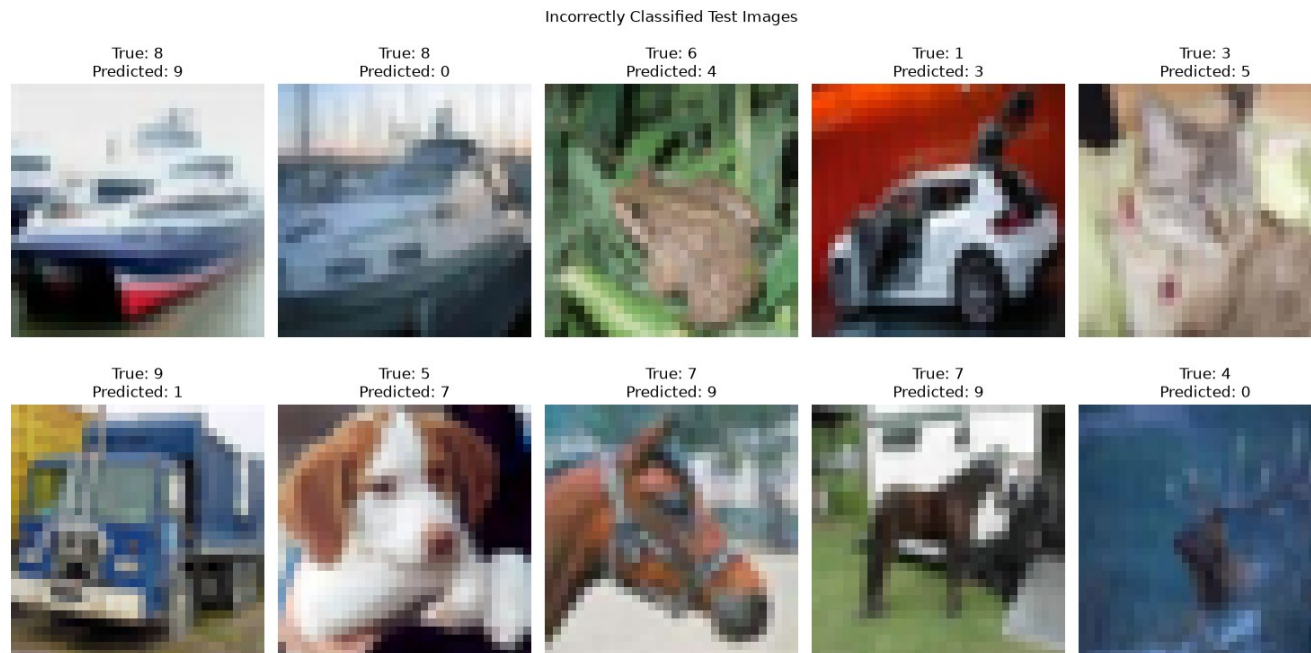
1.4837
test loss

2.1M
parameters

The test set was opened exactly once, after every architecture and hyperparameter decision was final.

Part 17 - Where it fails

- The mistakes are concentrated between visually similar classes



● Hardest

cat 35.52% F1
dog 40.93%
bird 43.08%

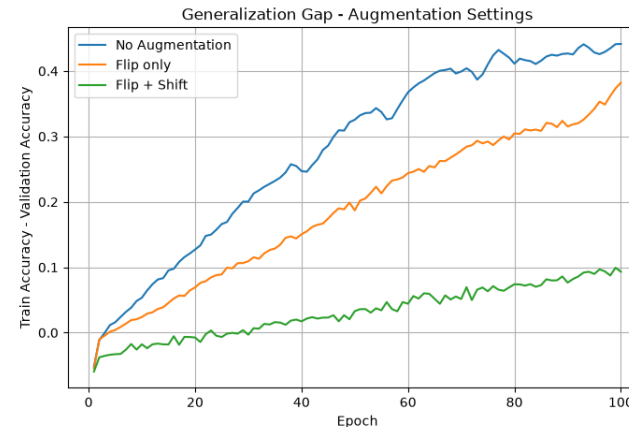
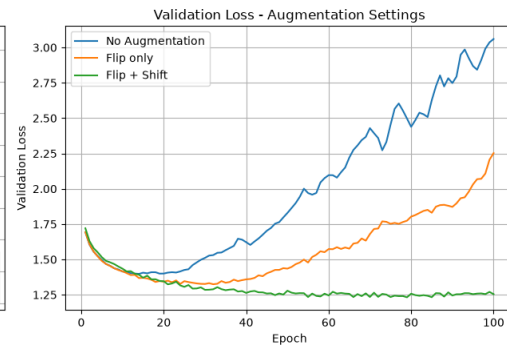
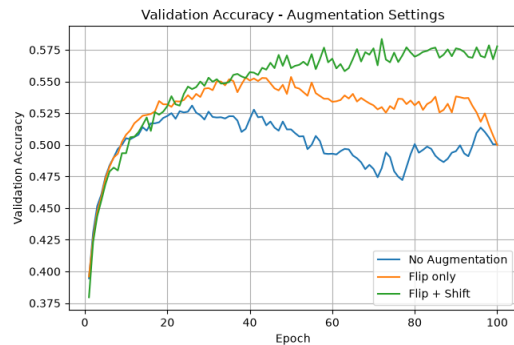
● Easiest

automobile 64.24% F1
ship 62.74%
horse 59.03%

- Cat and dog account for 350 errors between them, the largest confusion pair in the matrix.

Extra - Data augmentation

- Random flip and shift, applied before the image is flattened



- The biggest gain
No augmentation
53.10%, gap 0.4414
- Flip only
55.36%, gap 0.3822
- Flip + Shift
58.36%, gap 0.0932

● +5.26 points and the gap nearly closed. The strongest single result in the project, and it cost no parameters.

What moved the needle

● Six findings that shaped the result



01

Depth did almost nothing

0.76 points across 1, 3 and 5 layers



02

Width is where the parameters live
the first layer holds 75% of them



03

Every architecture overfit

training 82%, validation 52%



04

Regularization beat architecture

Dropout closed the gap 30% to 6%



05

BatchNorm made it worse

99.13% train, 44.88% validation



06

Augmentation won outright

58.36%, the best score recorded



The constraint in the brief mattered more to the final number than any hyperparameter chosen after it.

Part 20 - Why this needs a CNN

● The ceiling is architectural, not a tuning failure

● What flattening destroys

32 x 32 x 3 becomes 3,072 x 1

All pixel values survive, but their arrangement does not

The model never learns that two pixels are neighbours

● What that costs

Edges, corners and textures have to be relearned at every position

A cat three pixels to the left is a different input vector

This is why cat and dog collapse into each other

● What a CNN gives back

The same filter applied everywhere

Local patterns learned once and reused across the image

Hierarchical features, from edges to shapes to objects

Position no longer matters

● Our augmentation result points the same way: the fix was more spatial variety, not more parameters.

WHAT WE TOOK AWAY

The architecture set the ceiling, not the hyperparameters.



Hardest

telling a real gain from validation-set noise



Best judgement

selecting on the gap once accuracy tied



Next time

start with augmentation, not with depth

Thank you

Khalid • Wasan